GROUP NAME: CODE MONKEYS **SUBJECT:** IT 105 Information Management I

COURSE, YEAR & SECTION: BSIS 2A **PROFESSOR:** Dr. GUILLERMO V. RED, Jr.

Optimization Report

When we first ran our five chosen BI queries against the 100,000-row dataset, the numbers made us pause. Query 2 (Course Popularity) took **1.203 seconds** painfully slow for something that should feel instant. Looking at the execution plans, we quickly saw why: grouping by the text column courseName triggered a full table scan and filesort, with no index being used at all. Query 5 had the same problem, and Query 3 (Instructor Workload) also grouped on a text field while juggling three tables. Even Query 1 (Monthly Enrollments) couldn't take advantage of an index because YEAR() and MONTH() were wrapped around enrollmentDate. Only Query 4 (Average Students per Course) ran smoothly from the beginning because it aggregated directly on the already-indexed courseID.

Our first move was aggressive indexing. We created over 20 indexes in a matter of minutes but we did it blindly. We targeted columns that didn't even exist in the schema and accidentally duplicated indexes that primary keys and foreign keys already cover. The screenshot of our attempt is an honest record of that trial-and-error phase: a lot of 0 row(s) affected messages and wasted execution time with zero performance payoff. That experience taught us that more indexes don't equal better performance; they often just add clutter.

So we stepped back and cleaned house. We dropped every unnecessary index and kept only **one**: a simple index on enrollments(enrollmentDate). The real transformation came from **rewriting the queries** specifically, changing GROUP BY courseName to GROUP BY courseID, courseName, using COUNT(*) consistently, and reordering joins to start from the smallest table. These changes allowed MySQL to exploit existing foreign-key and primary-key indexes for aggregation and sorting without forcing expensive string comparisons.

Query	Before (s)	After (s)	Improvement
1 – Monthly Enrollments	0.313	0.235	24.9%
2 – Course Popularity	1.203	0.265	78.0%
3 – Instructor Workload	0.734	0.250	65.9%
4 – Avg Students/Course	0.375	0.234	37.6%
5 – High Enrollment	0.453	0.281	38.0%

Looking ahead, we're walking away with a few firm guidelines for large-scale systems: **(1)** always start with EXPLAIN, not with index creation; **(2)** group and join on indexed integer columns whenever possible leave text fields for display, not computation; **(3)** don't blindly index; foreign keys and unique constraints already bring their own; and **(4)** treat query design as the primary lever for speed. This phase was a real-world lesson that performance tuning is less about how many indexes you add and more about how well you understand your data and your queries.